Photo Manager — Project JSON Reference

Date: 24 Oct 26

Document Version: 1.0

Schema Version: 2
Status: Published
File Format: OpenDocument Text (ODT)
Reference File: `project.json`

---

1. Revision History

Document Version	Date	Description
1.0	20 Oct 26	Initial draft of the Project JSON reference for Schema Version 2. Documents the JSON structure, field definitions and a working Journi Importer example.

Table of Contents

1. Revision History....1
2. Purpose and Scope....3
3. Overall JSON Structure....3
4. Workspace Object....4
   4.1. Workspace Fields....4
   4.2. Field Definitions....4
      4.2.1. schema....4
      4.2.2. importer....4
      4.2.3. source....5
      4.2.4. source_id....5
      4.2.5. source_title....5
      4.2.6. registration_status....5
      4.2.7. download_status....6
      4.2.8. event....6
      4.2.9. image_folder....6
      4.2.10. created_at....6
5. Metadata Policies Object....7
   5.1. Policy Fields....7
   5.2. Policy Field Definitions....8
      5.2.1. propagate....8
      5.2.2. priority....8
      5.2.3. default....9
   5.3. Policy Definitions....9
      5.3.1. event....9
      5.3.2. year....9
      5.3.3. location....10
      5.3.4. info....10
6. Photo Object....11
   6.1. Photo Object Fields....11
   6.2. Photo Field Definitions....12
      6.2.1. guid....12
      6.2.2. moment_id....12
      6.2.3. lat....12
      6.2.4. lng....13
      6.2.5. file....13
      6.2.6. downloaded....13
      6.2.7. retry_count....14
      6.2.8. status....14
      6.2.9. exif_written....14
      6.2.10. last_updated....14
      6.2.11. location_key....15
      6.2.12. geo_location....15
      6.2.13. event....15
      6.2.14. year....16
      6.2.15. location....16
      6.2.16. info....16

6.3. Complete project.json Example — Journi Importer..... 17

2. Purpose and Scope

This document defines the structure and contents of the `project.json` file used by Photo Manager.

It serves as a reference for developers, describing the available fields, their data types, required or optional status, valid values, and the meaning of those values where clarification is necessary.

The document includes a complete example of a `project.json` file to illustrate the defined structure.

This reference describes the data format only. It does not describe application responsibilities, internal processing, or the procedures by which the file is created or updated.

3. Overall JSON Structure

The `project.json` file contains three top-level members:

- `workspace` — describes the project and its source.
- `metadata_policies` — defines the metadata policies associated with the project.
- `photos` — contains the collection of photographs and their associated data.

The overall structure is:

```
{
  "workspace": {},
  "metadata_policies": {},
  "photos": []
}
```

The objects and array shown above are structural placeholders. Their complete contents and field definitions are documented in the following sections.

All field names are case-sensitive. Field names, data types, and permitted values must conform to the definitions in this reference.

4. Workspace Object

The `workspace` object contains information describing the project as a whole. It identifies the project, records its source, provides project-wide metadata, and contains project status and configuration fields.

Every `project.json` file contains one `workspace` object.

4.1. Workspace Fields

The `workspace` object contains the following fields:

Field	Data Type	Required
`schema`	Integer	Yes
`importer`	String	Yes
`source`	String	Yes
`source_id`	String	Yes
`source_title`	String	Yes
`registration_status`	String	Yes
`download_status`	String	Yes
`event`	String	Yes
`image_folder`	String	Yes
`created_at`	String (UTC timestamp)	Yes

4.2. Field Definitions

4.2.1. `schema`

Data type: Integer
Required: Yes
Valid values: Positive integer.

Identifies the version of the `project.json` data structure. This value is checked by Photo Manager to determine if a project is compatible with the Photo Manager version that is being used.

4.2.2. `importer`

Data type: String
Required: Yes
Valid values: Non-empty importer identifier.

Identifies the Importer associated with the project.

4.2.3. `source`

Data type: String
Required: Yes
Valid values: Importer-defined string.

Identifies the external source associated with the project. Examples include `Journi`, `Filesystem`, and `Komoot`. The source field enables reconnection to the original source if necessary.

4.2.4. `source_id`

Data type: String
Required: Yes
Valid values: Importer-defined identifier.

Identifies the source object associated with the project. Its format depends on the source system. When combined with the `source` this permits the Importer to reestablish a connection to the original source photographs.

4.2.5. `source_title`

Data type: String
Required: Yes
Valid values: Non-empty string.

Contains a human-readable title associated with `source_id`. If the source does not provide a descriptive title, a suitable alternative may be used.

4.2.6. `registration_status`

Data type: String
Required: Yes
Valid values:

- `NOT_STARTED`
- `IN_PROGRESS`
- `COMPLETE`

Indicates the current registration status of the project's assets.

4.2.7. `download_status`

Data type: String
Required: Yes
Valid values:

- `NOT_STARTED`
- `IN_PROGRESS`
- `COMPLETE`

Indicates the status of the project's initial asset download phase. `COMPLETE` means that the phase has finished; it does not necessarily mean that every asset was downloaded successfully.

4.2.8. `event`

Data type: String
Required: Yes
Valid values: Any UTF-8 string.

Contains the project's default event name. Individual photo records may contain different event values.

4.2.9. `image_folder`

Data type: String
Required: Yes
Valid values: Valid filesystem directory name.

Identifies the directory name associated with the project's image assets. The value must be valid for the host filesystem. The directory name is relative to the location of `project.json`.

4.2.10. `created_at`

Data type: String
Required: Yes
Valid format: ISO 8601 UTC timestamp in the form:

`YYYY-MM-DDTHH:MM:SS.sssZ`

Records the date and time at which the project was created. The trailing `Z` indicates UTC.

5. Metadata Policies Object

The `metadata_policies` object contains project-wide policies for the metadata fields `event`, `year`, `location`, and `info`.

Each policy object contains a required propagation mode and may also contain a provider priority list and a default value.

5.1. Policy Fields

The `metadata_policies` object contains the following policy entries:

Field	Data Type	Required
`event`	Object	Yes
`year`	Object	Yes
`location`	Object	Yes
`info`	Object	Yes

Each policy object contains the following fields:

Field	Data Type	Required
`propagate`	String	Yes
`priority`	Array of strings	No
`default`	String	No

5.2. Policy Field Definitions

5.2.1. `propagate`

Data type: String

Required: Yes

Valid values:

- `event`
- `group`
- `location`
- `none`

Value	Meaning
`event`	Applies the value to all assets in the project.
`group`	Applies the value to assets sharing the same `moment_id`.
`location`	Applies the value to assets sharing the same `location_key`.
`none`	Disables automatic propagation of the value.

Defines the scope over which a metadata value is propagated.

Photo Manager refers to the `propagate` value when a photo is edited. The value determines which other photos, if any, are also affected by the edit.

The Importer should set an appropriate initial `propagate` value for each metadata policy. The user may subsequently change this value during a Photo Manager editing session.

5.2.2. `priority`

Data type: Array of strings

Required: No

Valid values: An ordered list of supported metadata provider identifiers, such as `user`, `importer`, `geocoder`, and `ai`.

Defines a priority order that the Importer may use when determining an appropriate initial value for the relevant metadata field of each photo. The order of entries in the array is significant.

Photo Manager does not use the `priority` field. It is optional and may be omitted from the policy object.

Example:

```
"priority": ["user", "geocoder", "ai"]
```

5.2.3. `default`

Data type: String

Required: No

Valid values: Any string, including an empty string (`""`).

Specifies a default value associated with the metadata field. Examples include `"Unknown"` and `""`.

This field may be used by the Importer when determining an initial metadata value for a photo. It is not used by Photo Manager and may be omitted from the policy object.

5.3. Policy Definitions

5.3.1. `event`

Data type: Object

Required: Yes

Defines the policy for the `event` metadata field.

Example:

```
"event": {
    "propagate": "event",
    "priority": ["user"],
    "default": ""
}
```

5.3.2. `year`

Data type: Object

Required: Yes

Defines the policy for the `year` metadata field.

Example:

```
"year": {
    "propagate": "group",
    "priority": ["user", "importer"],
    "default": ""
}
```

5.3.3. `location`

Data type: Object

Required: Yes

Defines the policy for the `location` metadata field.

Example:

```
"location": {
    "propagate": "location",
    "priority": ["user", "geocoder", "ai"],
    "default": "Unknown"
}
```

5.3.4. `info`

Data type: Object

Required: Yes

Defines the policy for the `info` metadata field, which holds descriptive text associated with a photo.

Example:

```
"info": {
    "propagate": "group",
    "priority": ["user", "ai"],
    "default": ""
}
```

6. Photo Object

The `photos` array contains one object for each asset managed by the project.

Each Photo Object records information associated with a single asset. It includes source identifiers, asset details, download and processing status, geographic information, and editable metadata.

Each Photo Object describes one asset only. Project-wide configuration is stored separately in the `workspace` and `metadata_policies` objects.

6.1. Photo Object Fields

Each Photo Object contains the following required fields:

Field	Data Type	Required
`guid`	String	Yes
`moment_id`	String	Yes
`lat`	Number (decimal) or null	Yes
`lng`	Number (decimal) or null	Yes

`file`	String	Yes
`downloaded`	Boolean	Yes
`retry_count`	Integer	Yes
`status`	String	Yes
`exif_written`	Boolean	Yes
`last_updated`	String (UTC timestamp)	Yes
`location_key`	String or null	Yes
`geo_location`	String or null	Yes
`event`	String	Yes
`year`	String	Yes
`location`	String	Yes
`info`	String	Yes

6.2. Photo Field Definitions

The following sections define the individual fields contained within a Photo Object.

6.2.1. `guid`

Data type: String

Required: Yes

Valid values: Importer-defined identifier.

Identifies the asset within the originating source.

The `guid` uniquely identifies the asset within the originating source. Its format and meaning are defined by the Importer. Depending on the source, it may identify an individual image or a group of related image files representing different resolutions of the same asset.

The identifier must be suitable for distinguishing the relevant source asset from other assets in that source.

The value may be used by the Importer when acquiring the photo asset.

Photo Manager treats the value as an identifier and does not interpret its internal format.

6.2.2. `moment_id`

Data type: String

Required: Yes

Valid values: Importer-defined identifier.

Identifies the logical group to which the asset belongs within the originating source.

The meaning of the grouping is defined by the Importer. Examples include a Journi Moment, a filesystem folder, a Komoot Highlight, or another source-specific grouping.

The `moment_id` is used by the `group` propagation policy to identify photos that belong to the same group. If the source does not provide natural groupings, the Importer should assign the same `moment_id` to all photos in the project.

6.2.3. `lat`

Data type: Number (decimal) or null

Required: Yes

Valid values: Decimal latitude in degrees, from `-90` to `+90`, or `null` if unavailable.

Stores the latitude associated with the asset.

When coordinates are available, the value represents the asset's geographic latitude. When they are unavailable, the field must still be present with the value `null`.

6.2.4. `lng`

Data type: Number (decimal) or null

Required: Yes

Valid values: Decimal longitude in degrees, from `-180` to `+180`, or `null` if unavailable.

Stores the longitude associated with the asset.

When coordinates are available, the value represents the asset's geographic longitude. When they are unavailable, the field must still be present with the value `null`.

Coordinate note: Latitude and longitude should be supplied as a pair. Photo Manager can open a location in Google Maps only when both values are available.

6.2.5. `file`

Data type: String

Required: Yes

Valid values: A valid filename for the host filesystem, unique within the project.

Identifies the filename under which the asset is stored within the project workspace.

The value must be the actual filename used for the asset in the project workspace. It must be unique within the project so that it does not conflict with another asset's filename.

Photo Manager uses the `file` value to display each photo during the edit process.

6.2.6. `downloaded`

Data type: Boolean

Required: Yes

Valid values: `true`, `false`.

Indicates whether the asset has been successfully downloaded and stored within the project workspace.

A value of `true` means the asset has been successfully stored. A value of `false` means the download has not been recorded as successful.

The field records download status only; it does not indicate whether metadata processing or editing is complete.

Import requirement: The Importer may use `downloaded: false` in its working records to track assets awaiting download or requiring retry. Only successfully downloaded photos, with `downloaded: true`, may be included in the project data passed to Photo Manager. Records for photos that fail to download must not be passed to Photo Manager.

6.2.7. `retry_count`

Data type: Integer

Required: `Yes`

Valid values: Integer greater than or equal to 0.

Records the number of retry attempts made after an unsuccessful download. The initial value is 0 and the count increases with each unsuccessful retry.

6.2.8. `status`

Data type: String

Required: Yes

Valid values: Core-defined workflow state, including `pending`, `downloaded` and `complete`.

Records the current workflow state of the asset within the project.

The value indicates the asset's current state in the Photo Manager workflow. The recognised values may be extended in future schema versions.

6.2.9. `exif_written`

Data type: Boolean

Required: Yes

Valid values: `true`, `false`.

Indicates whether the asset's editable metadata has been successfully written to its EXIF data.

A value of `true` means the metadata has been written successfully. A value of `false` means successful writing has not been recorded.

The initial value, supplied by the importer is `false`. Photo Manager may change the value to `true`.

6.2.10. `last_updated`

Data type: String (UTC timestamp)

Required: Yes

Valid values: ISO 8601 timestamp in the format `YYYY-MM-DDTHH:MM:SS.sssZ`.

Records the date and time the Photo Object was last modified.

The timestamp must be recorded in Coordinated Universal Time (UTC) and include the trailing `Z` timezone designator.

6.2.11. `location_key`

Data type: String or null

Required: Yes

Valid values: A location identifier or `null` if unavailable.

Identifies a logical geographic location shared by one or more photos within the project.

Photos that represent the same geographic location share the same `location_key`. The identifier's format is implementation-defined and should be treated as an opaque value.

The value may be `null` when geographic coordinates are unavailable or a location identifier cannot be determined.

Photo Manager uses the value when determining propagation behaviour.

6.2.12. `geo_location`

Data type: String or null

Required: Yes

Valid values: Any UTF-8 string or `null` if unavailable.

Stores a geographic description associated with the photo, determined by the Importer. The description may be derived from geographic coordinates using reverse geocoding or another suitable source.

The Importer may use this value to determine the initial value of the user-editable `location` field. The two fields serve different purposes: `geo_location` records the importer-determined description, while `location` contains the location metadata that the user can review and edit.

The value may be `null` if a geographic description cannot be determined.

6.2.13. `event`

Data type: String

Required: Yes

Valid values: Any UTF-8 string, including an empty string.

Stores the event or occasion associated with the photo.

The value is user-editable metadata. Its initial value may be supplied by the Importer based on information available from the source.

6.2.14. `year`

Data type: String

Required: Yes

Valid values: Any UTF-8 string, including an empty string.

Stores a temporal descriptor associated with the photo. The value is not restricted to a calendar year; it may represent any period, time of day or temporal grouping that is meaningful to the user.

For example, in a project titled *Janes Wedding*, values might include `Morning`, `Afternoon`, `Evening` or `Wee Hours`. For a project documenting a holiday, the value might instead be a calendar year such as `2025`.

The value is user-editable metadata. Its initial value may be supplied by the Importer based on information available from the source.

6.2.15. `location`

Data type: String

Required: Yes

Valid values: Any UTF-8 string, including an empty string.

Stores the user-editable location metadata associated with the photo.

The Importer may use `geo_location` or other available source information to determine the initial value. The user can review and modify this value to provide the preferred location description.

The value is not required to match `geo_location`; it represents the location description chosen by the user.

6.2.16. `info`

Data type: String

Required: Yes

Valid values: Any UTF-8 string, including an empty string.

Stores additional descriptive information or comments associated with the photo.

The value is user-editable metadata. Its initial value may be supplied by the Importer based on information available from the source. The user can review and modify the text to preserve or add contextual information about the photo.

6.3. Complete project.json Example — Journi Importer

The following example is taken from a `project.json` file recently generated by the Journi Importer and successfully used with Photo Manager.

It demonstrates the complete structure of a Schema Version 2 project, including the `workspace` object, `metadata_policies` object and `photos` array.

The example contains six successfully downloaded photos. Values such as identifiers, coordinates, timestamps, metadata policies and descriptions are specific to this test project. They illustrate valid working data rather than mandatory values for every project.


```
{
    "workspace": {
        "schema": 2,
        "importer": "journi_importer/0.1.0",
        "source": "journi",
        "source_id": "geoffs-test",
        "source_title": "Geoff's test",
        "registration_status": "COMPLETE",
        "download_status": "COMPLETE",
        "event": "Import Test 2",
        "image_folder": "photos",
        "created_at": "2026-10-18T04:28:25.248Z"
    },
    "metadata_policies": {
        "event": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
```

```
        "year": {
            "propagate": "none",
            "priority": [],
            "default": ""
        },
        "location": {
            "propagate": "location",
            "priority": [],
            "default": ""
        },
        "info": {
            "propagate": "group",
            "priority": [],
            "default": ""
        }
    },
```

```
"photos": [
   {
       "guid": "1ijdmOxMuBTnSPnG",
       "moment_id": "112426436",
       "lat": -26.655161666666668,
       "lng": 153.09512216666667,
       "file": "1ijdmOxMuBTnSPnG.jpg",
       "downloaded": true,
       "retry_count": 0,
       "status": "downloaded",
       "exif_written": false,
       "last_updated": "2026-10-18T04:28:31.544Z",
       "location_key": "-26.66,153.10",
       "geo_location": "Sunshine Coast",
       "event": "Import Test 2",
       "year": "2026",
       "location": "Sunshine Coast",
       "info": "Dressed for dinner"
   },
```

```
 {
    "guid": "1ijdcyEHDgbXUbKt",
    "moment_id": "112426436",
    "lat": -26.655161666666668,
    "lng": 153.09512216666667,
    "file": "1ijdcyEHDgbXUbKt.jpg",
    "downloaded": true,
    "retry_count": 0,
    "status": "downloaded",
    "exif_written": false,
    "last_updated": "2026-10-18T04:28:31.855Z",
    "location_key": "-26.66,153.10",
    "geo_location": "Sunshine Coast",
    "event": "Import Test 2",
    "year": "2026",
    "location": "Sunshine Coast",
    "info": "Dressed for dinner"
},
```

```
 {
    "guid": "1ijcNgQnfmZfpaLY",
    "moment_id": "112426435",
    "lat": -33.26356166666667,
    "lng": 151.42956666666666,
    "file": "1ijcNgQnfmZfpaLY.jpg",
    "downloaded": true,
    "retry_count": 0,
    "status": "downloaded",
    "exif_written": false,
    "last_updated": "2026-10-18T04:28:33.729Z",
    "location_key": "-33.26,151.43",
    "geo_location": "New South Wales",
    "event": "Import Test 2",
    "year": "2025",
    "location": "New South Wales",
    "info": "Family shot"
},
```

```
{
    "guid": "1ilySQMA74h0DlnH",
    "moment_id": "112427022",
    "lat": -27.561611666666668,
    "lng": 151.95065333333332,
    "file": "1ilySQMA74h0DlnH.jpg",
    "downloaded": true,
    "retry_count": 0,
    "status": "downloaded",
    "exif_written": false,
    "last_updated": "2026-10-18T04:28:34.949Z",
    "location_key": "-27.56,151.95",
    "geo_location": "Toowoomba",
    "event": "Import Test 2",
    "year": "2024",
    "location": "Toowoomba",
    "info": "Maise. At Christmas time"
},
```

```
{
    "guid": "1ilyHLKJmQD7YQiU",
    "moment_id": "112427022",
    "lat": -27.561611666666668,
    "lng": 151.95065333333332,
    "file": "1ilyHLKJmQD7YQiU.jpg",
    "downloaded": true,
    "retry_count": 0,
    "status": "downloaded",
    "exif_written": false,
    "last_updated": "2026-10-18T04:28:36.329Z",
    "location_key": "-27.56,151.95",
    "geo_location": "Toowoomba",
    "event": "Import Test 2",
    "year": "2024",
    "location": "Toowoomba",
    "info": "Maise. At Christmas time"
},
```

```
        {
            "guid": "1ije93v1fGCBp82g",
            "moment_id": "112426441",
            "lat": -33.26358333333334,
            "lng": 151.42956333333333,
            "file": "1ije93v1fGCBp82g.jpg",
            "downloaded": true,
            "retry_count": 0,
            "status": "downloaded",
            "exif_written": false,
            "last_updated": "2026-10-18T04:28:37.139Z",
            "location_key": "-33.26,151.43",
            "geo_location": "New South Wales",
            "event": "Import Test 2",
            "year": "2024",
            "location": "New South Wales",
            "info": "Dad’s new motorcycle. Enfield bike"
        }
    ]
}
```